

Desafio Técnico

Construção de um Pipeline Híbrido de Modernização SQL → Python

Objetivo

Avaliar habilidades técnicas em desenvolvimento de software, integração de sistemas aplicada a um cenário real de modernização de sistemas legados. O candidato deverá projetar e implementar um pipeline híbrido (LLM + Rules) e apresentar a solução de forma clara, fundamentada e reproduzível.

Instruções Gerais

1. O candidato terá até 2 dias corridos (take-home) a partir do recebimento do desafio para concluir a entrega.
2. É permitido pesquisar e utilizar bibliotecas externas, desde que sejam justificadas no README.
3. É permitido o uso de assistentes de IA (Copilot, Claude, ChatGPT, Cursor etc.), desde que o candidato saiba defender cada decisão técnica na entrevista de revisão.
4. O resultado final deve ser entregue como repositório Git (público, privado com acesso concedido, ou em arquivo .zip).
5. Inclua um arquivo README.md contendo:
 - Descrição da pipeline desenvolvida e do fluxo de modernização proposto.
 - Passos para executar e testar a aplicação localmente (incluindo Docker Compose, se aplicável).
 - Explicação das decisões técnicas tomadas, com trade-offs considerados.
 - Limitações conhecidas e o que seria feito com mais tempo.

Descrição do Desafio

Cenário

A solução de modernização da empresa é composta por pipelines de modernização de sistemas legados. As pipelines híbridas etapas baseadas em chamadas a LLM e etapas determinísticas. Um dos volumes mais frequentes da área de inovação é a migração de lógica de negócio implementada em stored procedures de bancos legados — escritas em PL/pgSQL, T-SQL ou PL/SQL — para serviços modernos em Python. Você foi solicitado a criar uma nova pipeline, responsável pela modernização de stored procedures PL/pgSQL para Python 3.14.

O pipeline recebe o código de uma stored procedure (e, opcionalmente, o schema das tabelas referenciadas) e produz, ao final do fluxo, um módulo Python 3.14 equivalente, junto com um relatório das decisões e validações realizadas. O foco está na qualidade do desenho da pipeline e nas decisões de tradução, não na cobertura sintática total da linguagem.

Os anexos deste documento (**A a F**) fornecem o material de entrada: o schema do banco legado e cinco stored procedures de complexidade crescente. O candidato deve usá-las como casos de teste da pipeline.

A Pipeline Híbrida

A pipeline deve implementar uma pipeline com pelo menos quatro etapas, orquestradas como nós de um grafo:

6. **Parsing** — Receber o código SQL e produzir uma representação estruturada (AST, árvore de tokens classificados ou estrutura intermediária equivalente). É aceitável e recomendado o uso de bibliotecas de parsing SQL (sqlglot, pglast, sqlparse), desde que a escolha seja justificada.
7. **Análise semântica** — Identificar construções relevantes (parâmetros IN/OUT, variáveis, cursores, transações, exceções, CTEs, chamadas a outras funções) e marcar pontos de risco para a tradução (ex.: cursores que podem virar gargalo, RAISE, FOR UPDATE, JSONB, recursão).
8. **Geração** — Produzir código Python 3.14 equivalente. O uso de LLM nesta etapa é permitido e bem-vindo, desde que o prompt e o contexto sejam construídos a partir das saídas das etapas anteriores (não basta enviar a procedure bruta direto para o modelo). A escolha entre delegar a query original ao SGBD via texto SQL ou reescrever a lógica em Python puro é uma decisão arquitetural que o candidato deve justificar.
9. **Validação** — Verificar a saída gerada. Como mínimo, executar verificações estáticas (parsing do Python gerado com `ast.parse`, linting). Como evolução desejada, executar testes ou comparar o comportamento contra a procedure original em um banco de teste.

Requisitos Técnicos Obrigatórios

1. Backend do Pipeline Híbrido

- O híbrido deve ser exposto como um endpoint de um servidor local com [langgraph cli](#) com, no mínimo, os seguintes endpoints:
 - **POST /modernize** — Recebe uma stored procedure SQL e retorna o código Python 3.14 equivalente, junto com o relatório das etapas (parsing, análise, geração, validação).
 - **GET /health** — Retorna o status do pipeline (ex.: `{"status": "ok"}`).
- Utilize boas práticas de organização e modularização (separação clara entre camada de API, orquestração do grafo, nós da pipeline, persistência e integrações externas).

2. Orquestração com LangGraph

- A pipeline deve ser orquestrada com **LangGraph**. Cada etapa (parsing, análise semântica, geração, validação) deve ser modelada como um nó do grafo, com estado tipado.
- Documente o desenho do grafo no README (diagrama textual ou imagem).

3. Banco de Dados

- Configure um banco de dados **PostgreSQL** para persistência do informações do pipeline.
- Crie uma tabela `modernization_history` com, no mínimo, os campos:
 - `id` — identificador único.
 - `source_code` — stored procedure SQL enviada.
 - `generated_code` — código Python 3.14 produzido.
 - `report` — relatório estruturado das etapas (JSONB).
 - `status` — desfecho da execução (sucesso, falha, parcial).
 - `created_at` — timestamp da execução.
- Toda execução da pipeline deve ser persistida nessa tabela, independentemente do desfecho.

4. Escalabilidade e Boas Práticas

- Use uma arquitetura que suporte crescimento futuro (volume de requisições, novos dialetos SQL, novos modelos de LLM).

Requisitos Bônus

Os bônus não são obrigatórios, implemente pelo menos um se desejar diferenciar a entrega.

Bônus 1 — Observabilidade de execução da pipeline híbrida

- Integre **Langfuse** (ou langsmith, prioritariamente langfuse) ao servidor langgraph para rastrear cada execução da pipeline (traces por execução, spans por nó do grafo, custo e latência das chamadas de LLM, se aplicável).
- Pode ser usado o Langfuse self-hosted via Docker. Documente a escolha.
- Apresente no README com a captura de tela com os tracings de execução dentro do langfuse ou langsmith evidenciando que o serviço de observabilidade está integrado ao pipeline.

Bônus 3 — QA

- Projeto passando em checks de qualidade estática (o padrão de verificações pode ser definido pelo candidato).
- Adicionar cobertura de testes com pytest.

Bônus 3 — Métrica de Evaluation do Pipeline

- Defina e implemente pelo menos uma métrica de avaliação automática da qualidade da migração — por exemplo: taxa de código Python gerado que passa em `ast.parse`, percentual de execuções concluídas sem erro, similaridade estrutural entre AST de origem e destino, equivalência comportamental (mesmo input → mesmo output) contra um banco de teste, ou avaliação por LLM-as-judge contra critérios objetivos.
- Registre os resultados dessa métrica no Langfuse (scores) ou em uma tabela própria, e exponha um endpoint ou notebook que demonstre a métrica calculada sobre o conjunto de procedures dos Anexos B a F.
- Justifique a escolha da métrica: o que ela captura, o que ela deixa de fora, e como ela seria evoluída em produção.

Entrega

10. Um repositório Git público contendo:
 - Código completo da pipeline, organizado em módulos claros.
 - Scripts de banco de dados (criação da tabela `modernization_history` e migrações, se houver).
 - `docker-compose.yml` ou equivalente para subir servidor local + PostgreSQL (e Langfuse, se aplicável).
 - Resultados da execução do pipeline sobre as procedures dos Anexos B a F (código Python gerado e relatórios).
 - Arquivo `README.md` com explicações detalhadas e diagrama do grafo LangGraph.
11. Instruções claras de como rodar a aplicação localmente, incluindo variáveis de ambiente necessárias (chaves de LLM, conexão com Postgres, credenciais Langfuse).

Critérios de Avaliação

Critério	Peso	Descrição
Funcionamento Geral	30%	Pipeline executa de ponta a ponta e atende aos requisitos básicos.
Código e Estrutura	20%	Qualidade do código, modularização, testes e aderência a boas práticas Python.
Arquitetura do Pipeline	15%	Modelagem do grafo LangGraph, separação de nós, estado e fluxo de decisão.

Critério	Peso	Descrição
Banco de Dados	10%	Modelagem, inicialização, integração e manipulação correta do PostgreSQL.
Escalabilidade	10%	Propostas para crescimento futuro: cache, filas, paralelização.
Documentação	10%	Clareza e completude do README e das decisões técnicas.
Bônus	+15%	Observabilidade com Langfuse (ou langsmith) e métrica de evaluation do pipeline.

Observação: a soma dos critérios obrigatórios é 100%. Os bônus podem somar até +15 pontos percentuais adicionais à nota final.

Etapa Pós-Entrega

Após o envio, será agendada uma sessão de revisão técnica de aproximadamente 45 minutos com o time de inovação. Nessa sessão, o candidato apresentará a solução, conduzirá um walk-through pelo código e responderá a perguntas sobre decisões arquiteturais, trade-offs e como evoluiria a solução em produção.

Anexos — Material de Entrada para a Pipeline

Esta seção contém o material que a pipeline deve ser capaz de processar. O Anexo A descreve o schema do banco legado fictício; os Anexos B a F apresentam cinco stored procedures de complexidade crescente que servem como casos de teste obrigatórios da pipeline.

Mapa de complexidade dos anexos

Anexo	Procedure	Complexidade	Construções principais
B	fn_saldo_cliente	Baixa	Função escalar, agregação, WHERE simples.
C	sp_atualizar_status_contas_inativas	Baixa-Média	Procedure, parâmetros IN/OUT, UPDATE em massa, GET DIAGNOSTICS.
D	sp_transferir_entre_contas	Média	Transação explícita, validação, EXCEPTION, RAISE, múltiplos UPDATE/INSERT.
E	sp_processar_lote_taxas	Alta	Cursor explícito, LOOP, CASE, JSONB, múltiplas tabelas, log de auditoria.
F	sp_relatorio_mensal_cliente	Muito Alta	CTE recursiva, função aninhada, RETURN QUERY (SETOF), RAISE NOTICE, EXCEPTION em camadas.

Anexo A — Schema do banco legado

O schema abaixo representa o banco de dados legado fictício de onde se originam as stored procedures dos anexos seguintes. Não é necessário instanciá-lo para o funcionamento do pipeline, mas ele deve ser fornecido ao pipeline como contexto opcional na etapa de geração para enriquecer o prompt do LLM e permitir validações mais sofisticadas.

```
-- =====
-- Schema do banco legado de referencia
-- Dominio: nucleo bancario simplificado
-- =====

CREATE TABLE clientes (
  id          BIGSERIAL PRIMARY KEY,
  nome        VARCHAR(200) NOT NULL,
  cpf         CHAR(11) NOT NULL UNIQUE,
  data_cadastro  TIMESTAMP NOT NULL DEFAULT NOW(),
  status      VARCHAR(20) NOT NULL DEFAULT 'ATIVO'
  CHECK (status IN ('ATIVO', 'INATIVO', 'BLOQUEADO'))
);

CREATE TABLE contas (
  id          BIGSERIAL PRIMARY KEY,
  cliente_id  BIGINT NOT NULL REFERENCES clientes(id),
  agencia     VARCHAR(10) NOT NULL,
  numero      VARCHAR(20) NOT NULL,
  tipo        VARCHAR(20) NOT NULL
  CHECK (tipo IN ('CORRENTE', 'POUPANCA', 'SALARIO')),
  saldo       NUMERIC(18,2) NOT NULL DEFAULT 0,
  status      VARCHAR(20) NOT NULL DEFAULT 'ATIVA'
  CHECK (status IN ('ATIVA', 'INATIVA', 'ENCERRADA')),
  data_abertura  TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE (agencia, numero)
);

CREATE TABLE transacoes (
  id          BIGSERIAL PRIMARY KEY,
  conta_origem_id  BIGINT REFERENCES contas(id),
  conta_destino_id BIGINT REFERENCES contas(id),
  tipo        VARCHAR(20) NOT NULL
  CHECK (tipo IN ('DEPOSITO', 'SAQUE', 'TRANSFERENCIA', 'TARIFA')),
  valor       NUMERIC(18,2) NOT NULL CHECK (valor > 0),
  data_transacao  TIMESTAMP NOT NULL DEFAULT NOW(),
  status      VARCHAR(20) NOT NULL DEFAULT 'EFETIVADA'
  CHECK (status IN ('EFETIVADA', 'CANCELADA', 'ESTORNADA'))
);

CREATE TABLE taxas (
  id          BIGSERIAL PRIMARY KEY,
  tipo_operacao  VARCHAR(20) NOT NULL,
  percentual    NUMERIC(7,4) NOT NULL DEFAULT 0,
  valor_minimo  NUMERIC(18,2) NOT NULL DEFAULT 0,
  vigente_de    DATE NOT NULL,
  vigente_ate   DATE
);

CREATE TABLE log_auditoria (
  id          BIGSERIAL PRIMARY KEY,
  entidade     VARCHAR(50) NOT NULL,
```

```
    entidade_id    BIGINT,  
    acao           VARCHAR(50) NOT NULL,  
    detalhes        JSONB,  
    criado_em      TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_contas_cliente ON contas(cliente_id);  
CREATE INDEX idx_transacoes_origem ON transacoes(conta_origem_id, data_transacao);  
CREATE INDEX idx_transacoes_destino ON transacoes(conta_destino_id, data_transacao);  
CREATE INDEX idx_log_entidade ON log_auditoria(entidade, entidade_id);
```

Anexo B — fn_saldo_cliente (Complexidade: Baixa)

Função escalar simples. Serve como caso de aquecimento da pipeline e validação básica do fluxo. A tradução esperada é uma função Python que executa uma única query agregada e retorna um valor numérico.

```
-- =====
-- Anexo B: fn_saldo_cliente
-- Complexidade: Baixa
-- Retorna o saldo total consolidado de todas as contas ativas
-- de um cliente.
-- =====

CREATE OR REPLACE FUNCTION fn_saldo_cliente(p_cliente_id BIGINT)
RETURNS NUMERIC(18,2)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC(18,2);
BEGIN
    SELECT COALESCE(SUM(saldo), 0)
        INTO v_total
        FROM contas
        WHERE cliente_id = p_cliente_id
            AND status = 'ATIVA';

    RETURN v_total;
END;
$$;
```

Anexo C — sp_atualizar_status_contas_inativas (Complexidade: Baixa-Média)

Procedure com parâmetros IN/OUT e UPDATE em massa baseado em subquery. Introduz validação de parâmetro e uso de GET DIAGNOSTICS. A tradução exige decisão sobre como expor parâmetros de saída em uma função Python (tupla, dataclass, dicionário).

```
-- =====
-- Anexo C: sp_atualizar_status_contas_inativas
-- Complexidade: Baixa-Media
-- Marca como INATIVA toda conta que nao tenha movimentacao
-- ha mais de p_dias dias. Retorna a quantidade de contas afetadas.
-- =====

CREATE OR REPLACE PROCEDURE sp_atualizar_status_contas_inativas(
    IN p_dias      INT,
    OUT p_afetadas INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_dias IS NULL OR p_dias <= 0 THEN
        RAISE EXCEPTION 'Parametro p_dias deve ser positivo, recebido: %', p_dias;
    END IF;

    UPDATE contas c
        SET status = 'INATIVA'
    WHERE c.status = 'ATIVA'
        AND NOT EXISTS (
            SELECT 1
            FROM transacoes t
            WHERE (t.conta_origem_id = c.id OR t.conta_destino_id = c.id)
                AND t.data_transacao >= NOW() - (p_dias || ' days')::INTERVAL
        );

    GET DIAGNOSTICS p_afetadas = ROW_COUNT;

    INSERT INTO log_auditoria (entidade, acao, detalhes)
    VALUES (
        'contas',
        'INATIVACAO_LOTE',
        jsonb_build_object('dias', p_dias, 'afetadas', p_afetadas)
    );
END;
$$;
```

Anexo D — sp_transferir_entre_contas (Complexidade: Média)

Procedure transacional com bloqueio de linha (FOR UPDATE), múltiplas validações, RAISE EXCEPTION e bloco EXCEPTION para auditoria de falhas. A tradução exige decisão arquitetural sobre transação: usar SQLAlchemy com gerenciamento explícito, manter a transação no SGBD via texto SQL, ou delegar a lógica de validação ao Python e manter apenas a parte transacional no banco.

```
-- =====
-- Anexo D: sp_transferir_entre_contas
-- Complexidade: Media
-- Transfere um valor entre duas contas em transacao atomica.
-- Valida saldo, status das contas e registra a transacao.
-- =====

CREATE OR REPLACE PROCEDURE sp_transferir_entre_contas(
    IN p_conta_origem    BIGINT,
    IN p_conta_destino   BIGINT,
    IN p_valor           NUMERIC(18,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_saldo_origem    NUMERIC(18,2);
    v_status_origem   VARCHAR(20);
    v_status_destino  VARCHAR(20);
BEGIN
    IF p_valor IS NULL OR p_valor <= 0 THEN
        RAISE EXCEPTION 'Valor invalido para transferencia: %', p_valor;
    END IF;

    IF p_conta_origem = p_conta_destino THEN
        RAISE EXCEPTION 'Conta de origem e destino nao podem ser iguais';
    END IF;

    SELECT saldo, status INTO v_saldo_origem, v_status_origem
    FROM contas WHERE id = p_conta_origem FOR UPDATE;

    SELECT status INTO v_status_destino
    FROM contas WHERE id = p_conta_destino FOR UPDATE;

    IF v_saldo_origem IS NULL THEN
        RAISE EXCEPTION 'Conta de origem % nao encontrada', p_conta_origem;
    END IF;

    IF v_status_origem <> 'ATIVA' OR v_status_destino <> 'ATIVA' THEN
        RAISE EXCEPTION 'Ambas as contas precisam estar ATIVAS';
    END IF;

    IF v_saldo_origem < p_valor THEN
        RAISE EXCEPTION 'Saldo insuficiente: saldo=% valor=%', v_saldo_origem, p_valor;
    END IF;

    UPDATE contas SET saldo = saldo - p_valor WHERE id = p_conta_origem;
    UPDATE contas SET saldo = saldo + p_valor WHERE id = p_conta_destino;

    INSERT INTO transacoes (conta_origem_id, conta_destino_id, tipo, valor)
    VALUES (p_conta_origem, p_conta_destino, 'TRANSFERENCIA', p_valor);
```

```
INSERT INTO log_auditoria (entidade, entidade_id, acao, detalhes)
VALUES (
    'transacoes',
    NULL,
    'TRANSFERENCIA_OK',
    jsonb_build_object(
        'origem', p_conta_origem,
        'destino', p_conta_destino,
        'valor', p_valor
    )
);

EXCEPTION
    WHEN OTHERS THEN
        INSERT INTO log_auditoria (entidade, acao, detalhes)
        VALUES (
            'transacoes',
            'TRANSFERENCIA_ERRO',
            jsonb_build_object(
                'origem', p_conta_origem,
                'destino', p_conta_destino,
                'valor', p_valor,
                'erro', SQLERRM
            )
        );
        RAISE;

END;
$$;
```

Anexo E — sp_processar_lote_taxas (Complexidade: Alta)

Procedure com cursor explícito, loop, lookup condicional em tabela de taxas vigentes, CASE de regras de negócio e log estruturado em JSONB. Aqui o candidato precisa decidir como traduzir o cursor: tradução ingênua resulta em N+1 queries; tradução pensada usa carregamento em lote (pandas, bulk fetch, SQL set-based) e preserva a semântica. A discussão dessa decisão pesa significativamente para níveis pleno e sênior.

```
-- =====
-- Anexo E: sp_processar_lote_taxas
-- Complexidade: Alta
-- Percorre as transacoes efetivadas em uma data, calcula a tarifa
-- aplicavel conforme o tipo, registra a tarifa como uma nova
-- transacao do tipo TARIFA e mantem log detalhado em JSONB.
-- =====

CREATE OR REPLACE PROCEDURE sp_processar_lote_taxas(
    IN p_data_referencia DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    cur_transacoes CURSOR FOR
        SELECT id, conta_origem_id, tipo, valor
        FROM transacoes
        WHERE DATE(data_transacao) = p_data_referencia
        AND status = 'EFETIVADA'
        AND tipo <> 'TARIFA';

    v_id          BIGINT;
    v_origem      BIGINT;
    v_tipo        VARCHAR(20);
    v_valor       NUMERIC(18,2);
    v_taxa        NUMERIC(18,2);
    v_percentual  NUMERIC(7,4);
    v_minimo      NUMERIC(18,2);
    v_total_taxas NUMERIC(18,2) := 0;
    v_count       INT := 0;
BEGIN
    OPEN cur_transacoes;

    LOOP
        FETCH cur_transacoes INTO v_id, v_origem, v_tipo, v_valor;
        EXIT WHEN NOT FOUND;

        SELECT percentual, valor_minimo
        INTO v_percentual, v_minimo
        FROM taxas
        WHERE tipo_operacao = v_tipo
        AND vigente_de <= p_data_referencia
        AND (vigente_ate IS NULL OR vigente_ate >= p_data_referencia)
        ORDER BY vigente_de DESC
        LIMIT 1;

        IF v_percentual IS NULL THEN
            CONTINUE;
        END IF;
```

```
v_taxa := GREATEST(v_valor * v_percentual / 100.0, v_minimo);

CASE v_tipo
  WHEN 'TRANSFERENCIA' THEN v_taxa := v_taxa;
  WHEN 'SAQUE'           THEN v_taxa := v_taxa * 1.10;
  ELSE                   v_taxa := v_taxa * 0.90;
END CASE;

IF v_origem IS NOT NULL THEN
  UPDATE contas SET saldo = saldo - v_taxa WHERE id = v_origem;

  INSERT INTO transacoes (conta_origem_id, tipo, valor, status)
  VALUES (v_origem, 'TARIFA', v_taxa, 'EFETIVADA');

  INSERT INTO log_auditoria (entidade, entidade_id, acao, detalhes)
  VALUES (
    'transacoes', v_id, 'TARIFA_APLICADA',
    jsonb_build_object(
      'transacao_origem', v_id,
      'tipo_origem',      v_tipo,
      'valor_origem',    v_valor,
      'percentual',      v_percentual,
      'taxa_aplicada',   v_taxa
    )
  );

  v_total_taxes := v_total_taxes + v_taxa;
  v_count       := v_count + 1;
END IF;
END LOOP;

CLOSE cur_transacoes;

INSERT INTO log_auditoria (entidade, acao, detalhes)
VALUES (
  'lote_taxes', 'LOTE_PROCESSADO',
  jsonb_build_object(
    'data_referencia', p_data_referencia,
    'transacoes',      v_count,
    'total_taxes',     v_total_taxes
  )
);
END;
$$$;
```

Anexo F — sp_relatorio_mensal_cliente (Complexidade: Muito Alta)

Função set-returning com CTE recursiva, chamada a outra função do banco (fn_saldo_cliente do Anexo B), RAISE NOTICE para observabilidade e bloco EXCEPTION com retorno de fallback degradado. A tradução exige decisões em camadas: como expor um endpoint que retorna múltiplas linhas; como traduzir CTE recursiva (manter em SQL bruto, reescrever com loop Python, ou apoiar-se em SQLAlchemy Core); como tratar a função aninhada (inlining, chamada cruzada, importação); como propagar logs e fallback. Diferenciador para a faixa sênior.

```
-- =====
-- Anexo F: sp_relatorio_mensal_cliente
-- Complexidade: Muito Alta
-- Gera relatorio mensal de movimentacao de um cliente.
-- Usa CTE recursiva para encadear meses, chama fn_saldo_cliente
-- aninhada e retorna SETOF via RETURN QUERY. Captura excecao
-- com fallback degradado.
-- =====

CREATE OR REPLACE FUNCTION sp_relatorio_mensal_cliente(
    p_cliente_id BIGINT,
    p_data_inicio DATE,
    p_data_fim    DATE
)
RETURNS TABLE (
    mes_referencia    DATE,
    total_creditos    NUMERIC(18,2),
    total_debitos     NUMERIC(18,2),
    saldo_consolidado NUMERIC(18,2),
    qtd_transacoes    INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_saldo_atual NUMERIC(18,2);
BEGIN
    IF p_data_inicio > p_data_fim THEN
        RAISE EXCEPTION 'Periodo invalido: inicio % > fim %', p_data_inicio, p_data_fim;
    END IF;

    v_saldo_atual := fn_saldo_cliente(p_cliente_id);
    RAISE NOTICE 'Saldo atual do cliente %: %', p_cliente_id, v_saldo_atual;

    RETURN QUERY
    WITH RECURSIVE meses AS (
        SELECT DATE_TRUNC('month', p_data_inicio)::DATE AS mes
        UNION ALL
        SELECT (mes + INTERVAL '1 month')::DATE
        FROM meses
        WHERE mes < DATE_TRUNC('month', p_data_fim)
    ),
    movimento AS (
        SELECT
            DATE_TRUNC('month', t.data_transacao)::DATE AS mes,
            SUM(CASE WHEN t.conta_destino_id IN (
                SELECT id FROM contas WHERE cliente_id = p_cliente_id
            ) THEN t.valor ELSE 0 END) AS creditos,
            SUM(CASE WHEN t.conta_origem_id IN (
                SELECT id FROM contas WHERE cliente_id = p_cliente_id
            ) THEN t.valor ELSE 0 END) AS debitos,
```

```
        COUNT(*) AS qtd
    FROM transacoes t
    WHERE t.status = 'EFETIVADA'
        AND t.data_transacao >= p_data_inicio
        AND t.data_transacao < p_data_fim + INTERVAL '1 day'
        AND (
            t.conta_origem_id IN (SELECT id FROM contas WHERE cliente_id =
p_cliente_id)
            OR t.conta_destino_id IN (SELECT id FROM contas WHERE cliente_id =
p_cliente_id)
        )
    GROUP BY 1
)
SELECT
    m.mes                                AS mes_referencia,
    COALESCE(mv.creditos, 0)              AS total_creditos,
    COALESCE(mv.debitos, 0)              AS total_debitos,
    v_saldo_atual + COALESCE(mv.creditos, 0)
        - COALESCE(mv.debitos, 0)        AS saldo_consolidado,
    COALESCE(mv.qtd, 0)::INT              AS qtd_transacoes
FROM meses m
LEFT JOIN movimento mv ON mv.mes = m.mes
ORDER BY m.mes;

EXCEPTION
    WHEN OTHERS THEN
        RAISE WARNING 'Falha ao gerar relatorio: %. Retornando linha de fallback.',
SQLERRM;

RETURN QUERY
SELECT
    DATE_TRUNC('month', p_data_inicio)::DATE,
    0::NUMERIC(18,2),
    0::NUMERIC(18,2),
    COALESCE(v_saldo_atual, 0),
    0::INT;

END;
$;$
```

Boa prova!